Key concepts:

- Structured, progressive curriculum - CompSci concepts are introduced with intention throughout the game.
- Minimal extraneous load - Minimise cognitive load irrelevant to the learning goal.
- Pseudocode blocks - Coding concepts are presented as pseudocode ‘blocks’ that can be combined to determine the character’s behaviour.
- Top down platformer - A ‘top-down’ environment that incorporates the Z axis. This is easier to implement than a traditional 2D platformer, but still allows for more complex levels including verticality.

Progressive curriculum:

This concept is vital for an educational game. Breaking down basic CompSci concepts into fundamentals allows for a clear, structured learning path for users to follow.

Example:

1. Sequencing
2. Loops
3. Conditionals
4. Functions
5. Variables

Cognitive Load Theory:

CLT identifies **three types of mental effort**:

- Intrinsic Load: The inherent difficulty of the subject matter itself. For example, understanding loops is intrinsically more difficult than understanding sequencing. The progressive curriculum outlined above is designed to manage this load by introducing concepts in order of increasing complexity.
- Extraneous Load: The mental effort required to process information that is not relevant to the learning goal. A confusing user interface, unclear instructions, or complex syntax rules all contribute to extraneous load.
- Germane Load: The "desirable" mental effort that is directly applied to processing new information and constructing long-term knowledge (or "schemas").

For optimal learning, the goal is to free up as much of the user’s limited cognitive resources by negating extraneous load. This is done through informed design decisions within the app. For example, clear instructions and an intuitive interface help the user focus on the desired aspects of the game, while a block-based pseudocode system reduces the need to learn syntax and specific languages’ nuances.

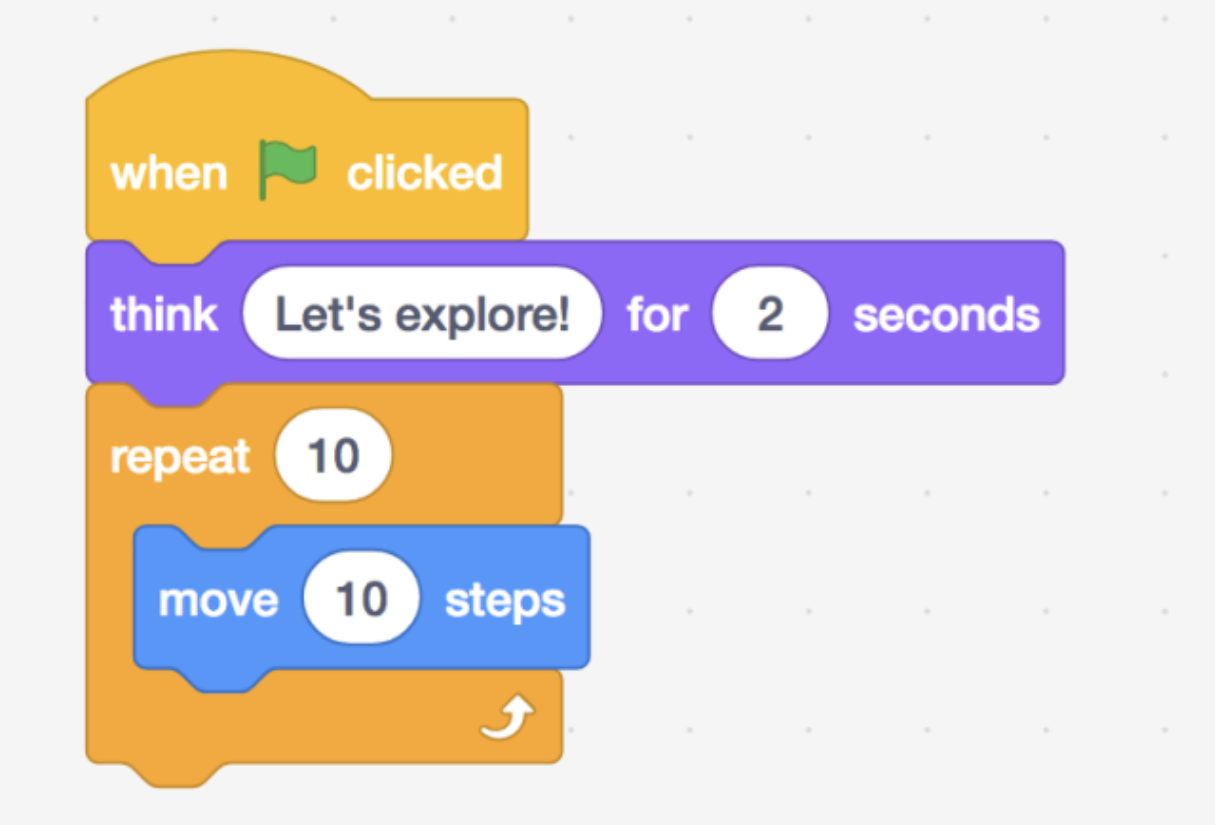
Block-based interface:

As mentioned above, this seems to me like the best implementation for the coding aspect of the game, as it would be much more simple to create than a text based system, and would likely be easier for users to adjust to. It is vital that this system is intuitive and informative. Real-time execution highlighting would enable users to see which ‘block’ of code is being executed for each action in the game, while robust error feedback would enhance the learning process. Limiting the blocks available for each level would solve the problem of users brute-forcing solutions (e.g. using 100 ‘move forward’ blocks, rather than looping).

Example block types:

- Movement blocks (e.g. move/turn forward/left/right, jump).
- Basic logic blocks (e.g. if x then… else…, while x do…, repeat x)
- Sensor blocks (e.g. obstacle ahead, gap below, obstacle movable, etc.)
- Function blocks
- Variable blocks

Example:



Top-down platformer:

This design approach blends the simplicity of a top-down maze approach with the dynamics of a traditional 2D platformer. It offers a 3D space to navigate, with the options to go under and over obstacles while only requiring very simple physics. This system creates the opportunity for robust progression and complex levels while creating an engaging environment for the user.

Top-down maze	**2D platformer**
👍Simple implementation, inherently logical. 👎May not be engaging for target audience.	👍Dynamic, engaging. 👎Complex implementation.

Example: